

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

Class Workshop (Terraform)

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: XXXXXXXXXXXXXXXX

BegutachterIn:

Wien, den 21. November 2025

Inhaltsverzeichnis

1 Überblick	1
2 Provisionierung der AWS-Ressourcen	1
2.0.1 Erstellung VPC	1
2.0.2 Erstellung Subnet	1
2.0.3 Erstellung Sicherheitsgruppe	2
2.0.4 EC2-Instanzen (Nodes)	3
2.0.5 Validierung	5
2.0.6 Docker Swarm Setup	6
2.0.7 Portainer Installation	8
2.0.8 Service-Deployment, Scale-Out und Scale-In	9
2.0.9 Kurzantworten	13
2.0.10 Anhang – Wichtige Befehle	13
3 Kubernetes Deployment Workshop mit Minikube	13
3.0.1 Installtion Minikube und Kubectl	14
3.0.2 Start und Vorbereitung	14
3.0.3 Deployment erstellen und skalieren	15
3.0.4 Testzugriff auf einen Pod	16
3.0.5 ClusterIP Service (intern)	17
3.0.6 NodePort Service (extern)	18
3.0.7 LoadBalancer Service (simuliert)	19
3.0.8 Vergleich der Service-Typen	21
3.0.9 YAML-Spezifikationen	22
3.0.10 Zusammenfassung	24

1 Überblick

Ziel: Provisionierung von AWS-Ressourcen, Aufbau eines Docker Swarm-Clusters, Installation von Portainer und Analyse von Skalierung und Ausfallszenarien.

2 Provisionierung der AWS-Ressourcen

2.0.1 Erstellung VPC

Neue VPC mit öffentlichem Subnet über den Assistenten im AWS-VPC-Dashboard erstellen. Security Group mit folgenden offenen Ports konfigurieren:

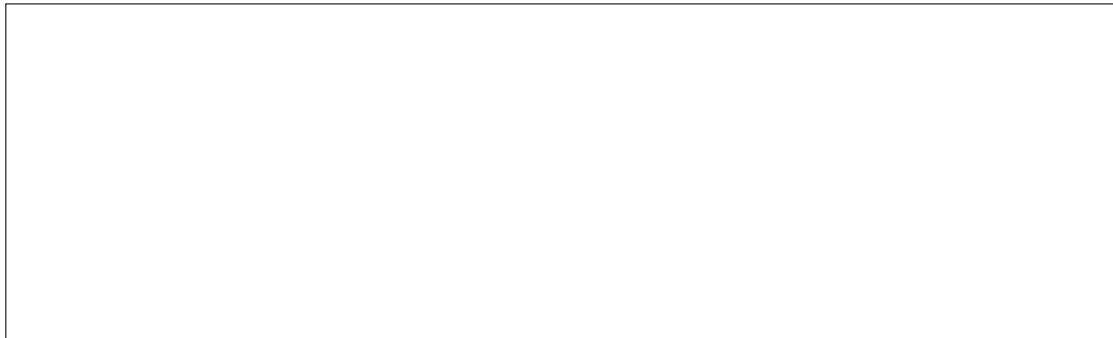


Abbildung 1: Einstellung der VPC inklusive manueller CIDR-Einstellung 10.0.0.0/16

2.0.2 Erstellung Subnet



Abbildung 2: Öffentliches Subnet mit Subnet-CIDR-Block 10.0.1.0/24



Abbildung 3: Automatische Vergabe der öffentlichen IPv4-Adresse aktiviert

2.0.3 Erstellung Sicherheitsgruppe

- 22/TCP – SSH (öffentlich)
- 80/TCP – HTTP (öffentlich)
- 2377/TCP, 7946/TCP/UDP, 4789/UDP – innerhalb der SG

Die Sicherheitsgruppe musste in zwei Schritten erstellt werden, damit sie den Vorgaben entspricht.

Schritt 1: Zuerst musste die Sicherheitsgruppe nur mit SSH und HTTP erstellt werden



Abbildung 4: Erstellung der Sicherheitsgruppe mit SSH und HTTP

Schritt 2: Erst nach dem Erstellen der Sicherheitsgruppe konnten wir für die Docker-Regeln die eigene Sicherheitsgruppe als Quelle angeben.

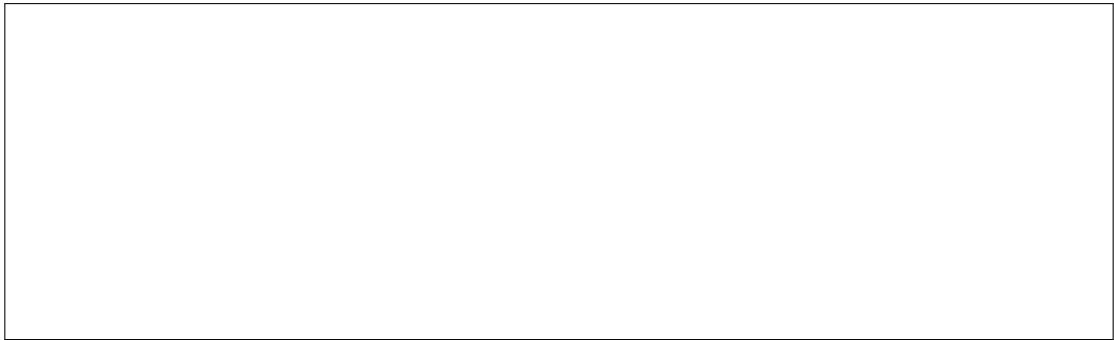


Abbildung 5: Anpassung der zuvor erstellten Sicherheitsgruppe mit zusätzlichen Regeln für Docker

2.0.4 EC2-Instanzen (Nodes)

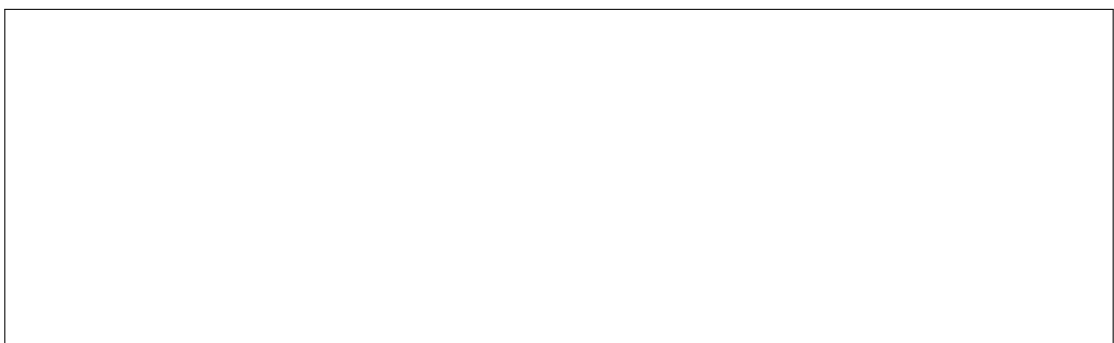


Abbildung 6: Erstellung der ersten Instanz in AWS



Abbildung 7: AMI: Ubuntu (neueste Version).



Abbildung 8: `t2.micro` (oder vergleichbar) als Instance Type



Abbildung 9: Aktivierung von **Auto-assign Public IP**.



Abbildung 10: Einfügen des User-Data-Skripts

```
#!/bin/bash
curl -o /home/ubuntu/install-docker.sh https://get.docker.com/
sh /home/ubuntu/install-docker.sh
```

Listing 2.1: EC2 User Data – Docker Installation



Abbildung 11: EC2 Launch – Übersicht der Einstellungen

2.0.5 Validierung

```
ssh -i <key.pem> ubuntu@<PUBLIC_IP>
docker --version
```



Abbildung 12: SSH-Verbindung und Docker-Version

2.0.6 Docker Swarm Setup

```
docker swarm init --advertise-addr <PRIVATE_IP>  
docker node ls
```



Abbildung 13: `docker swarm init` auf der ersten Instanz

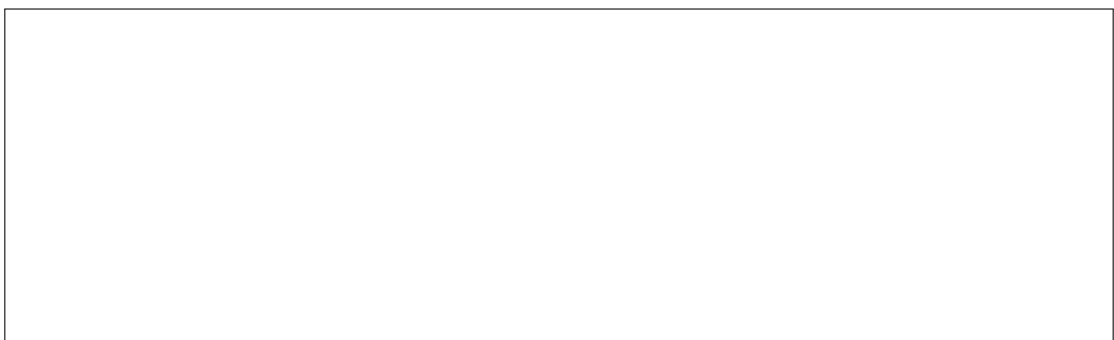


Abbildung 14: `docker node ls` auf einer Instanz



Abbildung 15: Hinzufügen von zwei Workern



Abbildung 16: Generieren des Manager-Tokens



Abbildung 17: Hinzufügen eines zweiten Managers



Abbildung 18: `docker node ls` mit 4 Knoten

2.0.7 Portainer Installation

```
curl -L https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o  
    portainer-agent-stack.yml  
docker stack deploy -c portainer-agent-stack.yml portainer
```



Abbildung 19: Portainer-Installation über die Kommandozeile

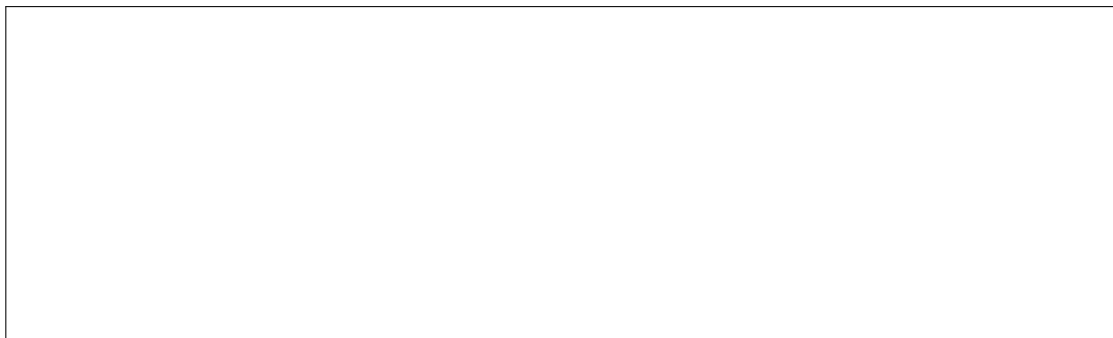


Abbildung 20: Erstellung eines neuen Logins für Portainer



Abbildung 21: Angezeigte Services nach der Installation von Portainer

2.0.8 Service-Deployment, Scale-Out und Scale-In

```
docker service create \  
  --name hello \  
  --replicas 3 \  
  --publish published=80,target=80 \  
  karthequian/helloworld:latest
```



Abbildung 22: Service mit 3 Replikas und Port 80 erstellen



Abbildung 23: Die 3 Replikas erhalten jeweils einen eigenen Slot



Abbildung 24: Die 3 Replikas werden auf die 4 verschiedenen Nodes in AWS verteilt



Abbildung 25: Hello-World-Seite nach Deployment des Service

```
docker service scale hello=10
```



Abbildung 26: Die 10 Replikas werden nach dem Hochskalieren auf verschiedene Slots verteilt



Abbildung 27: Die 10 Replikas werden auf die 4 verschiedenen Nodes verteilt



Abbildung 28: Hello-World-Seite wird nun von verschiedenen Containern bedient



Abbildung 29: Node wird als „down“ angezeigt, nachdem eine EC2 heruntergefahren wurde



Abbildung 30: Services auf dem betroffenen Node wechselten erst nach „ready“, dann nach „shutdown“, neue Replikas wurden auf andere Nodes verteilt

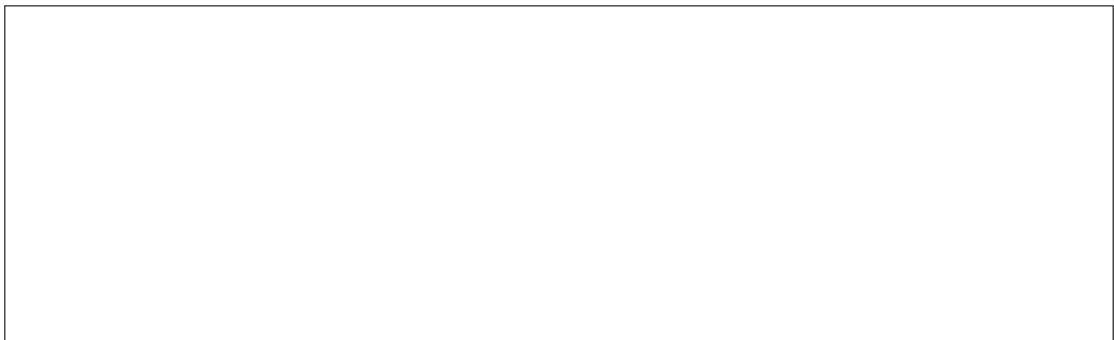


Abbildung 31: Node down mit zwei Services; auf den restlichen drei Nodes laufen insgesamt 10 Services

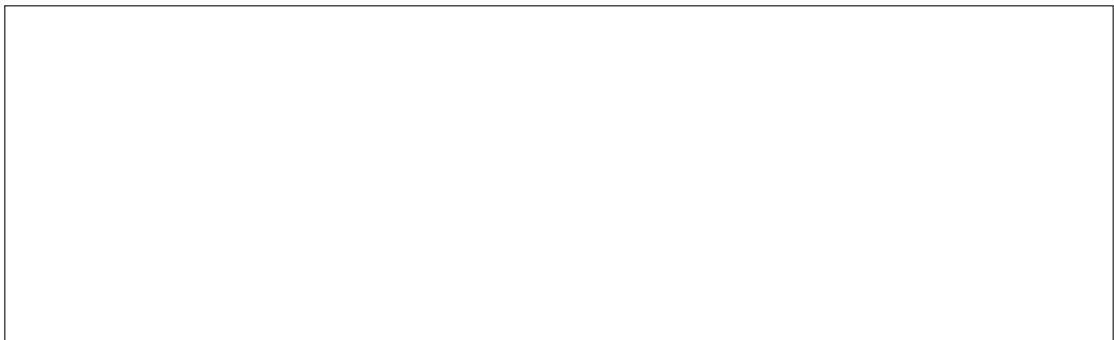


Abbildung 32: Nach dem Hochfahren wurden die übrigen Services wieder sauber auf alle 4 Nodes verteilt



Abbildung 33: Beim Scale-Down wurden die verbleibenden 3 Services erneut gleichmäßig verteilt

2.0.9 Kurzantworten

- **Skalierung des Clusters:**

Skalieren Sie Ihren Cluster von 3 auf 10 Maschinen.

Beobachtung: Beim Öffnen der Hello-World-Seite wird jeweils ein anderer Container angezeigt.

- **Wiederinbetriebnahme eines Workers:**

Wenn Sie den Worker wieder online schalten, werden die Services automatisch auf alle vier Nodes verteilt.

- **Erneute Skalierung:**

Bei einer erneuten Skalierung werden die drei Services erneut gleichmäßig auf die vier Nodes verteilt.

2.0.10 Anhang – Wichtige Befehle

```
docker service ls
docker service ps hello
docker node ls
docker service scale hello=10
```

3 Kubernetes Deployment Workshop mit Minikube

Ziel: In diesem Workshop wurde ein Kubernetes-Cluster mit `Minikube` erstellt, ein `nginx`-Deployment angelegt und verschiedene Service-Typen (ClusterIP, NodePort, LoadBalancer) getestet.

3.0.1 Installtion Minikube und Kubectl

```
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/  
minikube-linux-amd64  
sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-  
linux-amd64
```

Listing 3.1: Installation von Minikube

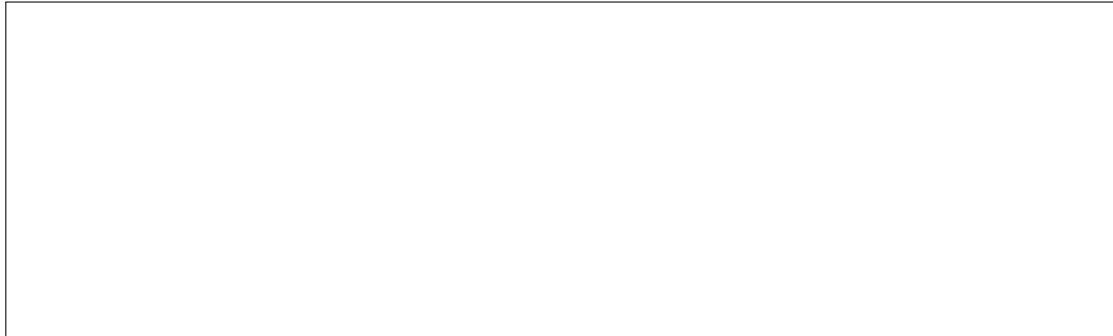


Abbildung 34: Download und Installtion von Minikube

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/  
stable.txt)/bin/linux/amd64/kubectl"  
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Listing 3.2: Installation von Kubectl

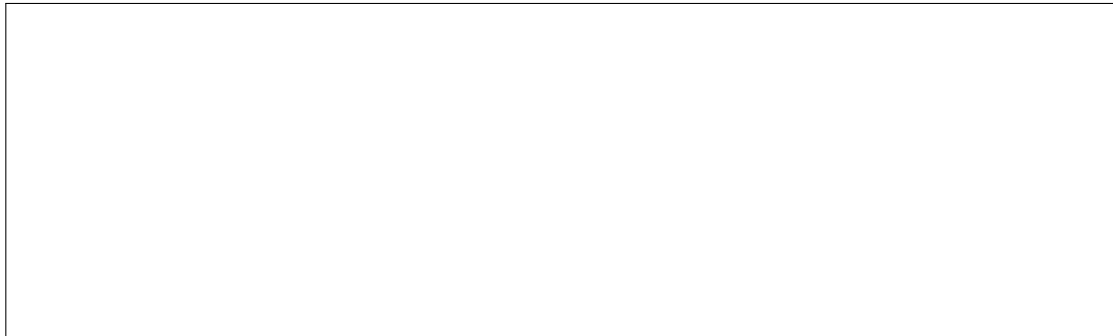


Abbildung 35: Download und Installation von Kubectl

3.0.2 Start und Vorbereitung

```
minikube start  
kubectl get nodes
```

Listing 3.3: Start von Minikube und Überprüfung der Umgebung



Abbildung 36: Start von Minikube mit einer einzelnen Node (2GB Ram da sonst alles gebraucht wird)



Abbildung 37: Alle laufenden Notes

3.0.3 Deployment erstellen und skalieren

```
kubectl create deployment nginx --image=nginx:stable  
kubectl get deployments  
kubectl get pods
```

Listing 3.4: nginx-Deployment mit einem Pod erstellen



Abbildung 38: Erstellung des nginx-Deployments



Abbildung 39: Anzeigen des erstellten Deployments

```
kubectl scale deployment nginx --replicas=4  
kubectl get pods
```

Listing 3.5: Deployment auf vier Replikas skalieren



Abbildung 40: Vier Replikas des nginx-Deployments sind aktiv

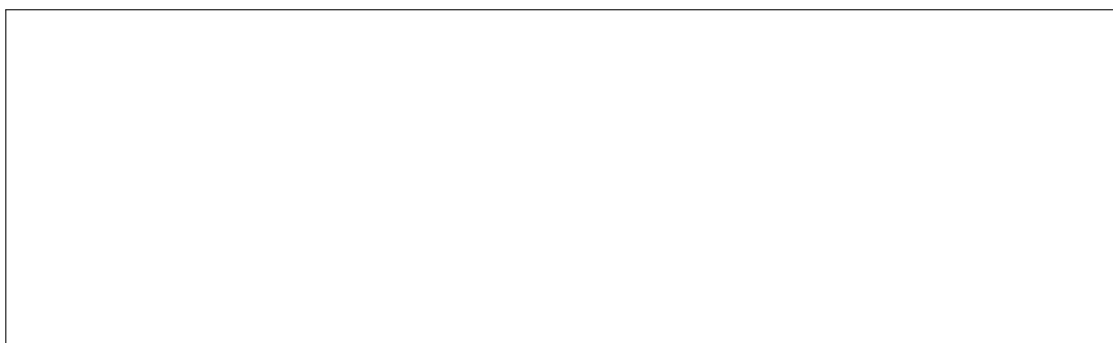


Abbildung 41: Anzeigen der erstellten Pods

3.0.4 Testzugriff auf einen Pod

```
kubectl expose deployment nginx --type=NodePort --Port=8080
```

Listing 3.6: Port-Forwarding für lokalen Zugriff



Abbildung 42: Erstelltes Dopleymnt auf Port 8080 exposen

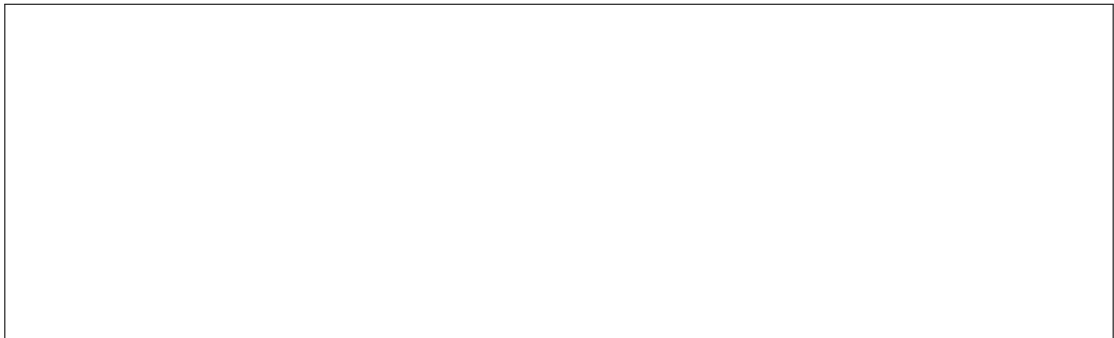


Abbildung 43: Browser öffnen mit Zugriff auf Pod

3.0.5 ClusterIP Service (intern)

```
kubectl expose deployment nginx \  
  --name=nginx-clusterip \  
  --type=ClusterIP \  
  --port=8080 \  
  --target-port=80  
kubectl get svc nginx-clusterip
```

Listing 3.7: Erstellung eines internen ClusterIP-Service



Abbildung 44: Interner ClusterIP-Service für das nginx-Deployment

```
kubectl run test-client --rm -it --image=curlimages/curl --restart=Never --  
  sh  
curl -I http://nginx-clusterip:8080
```

Listing 3.8: Test im Cluster über temporären Curl-Pod



Abbildung 45: Erfolgreicher Curl-Test des internen ClusterIP-Service

3.0.6 NodePort Service (extern)

```
kubectl expose deployment nginx \  
  --name=nginx-nodeport \  
  --type=NodePort \  
  --port=8080 \  
  --target-port=80  
kubectl get svc nginx-nodeport
```

Listing 3.9: Erstellung eines NodePort-Service



Abbildung 46: NodePort-Service mit zugewiesenem externen Port (z. B. 31245)

```
minikube service nginx-nodeport --url
```

Listing 3.10: Abrufen der externen URL über Minikube



Abbildung 47: Von Minikube generierte URL für NodePort-Zugriff

Zugriff im Browser oder mit `curl`:

```
curl -I $(minikube service nginx-nodeport --url)
```



Abbildung 48: Erfolgreicher Curl-Test des internen Nodeport-Service

3.0.7 LoadBalancer Service (simuliert)

```
kubectl expose deployment nginx \  
  --name=nginx-lb \  
  --type=LoadBalancer \  
  --port=8080 \  
  --target-port=80
```

Listing 3.11: Erstellung eines LoadBalancer-Service

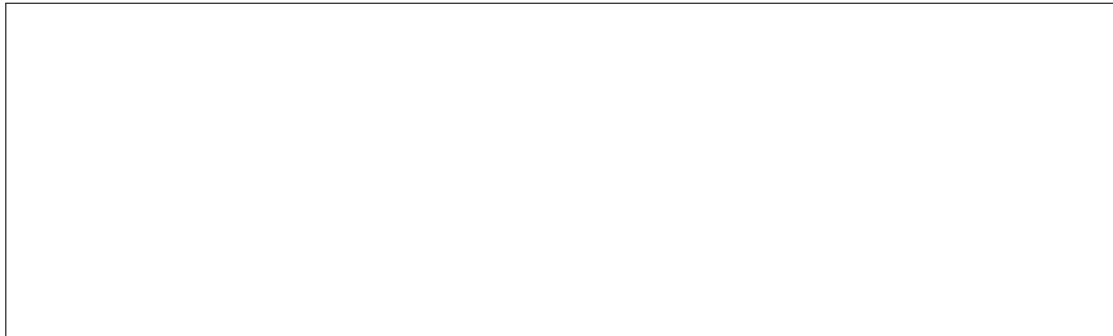


Abbildung 49: LoadBalancer-Service mit zugewiesenem externen Port (z. B. 31245)

Da Minikube keinen echten Cloud-Load-Balancer bereitstellt, wird die Funktion über einen lokalen Tunnel simuliert.

```
minikube tunnel
```

Listing 3.12: Start des Minikube-Tunnels (neues Terminal)

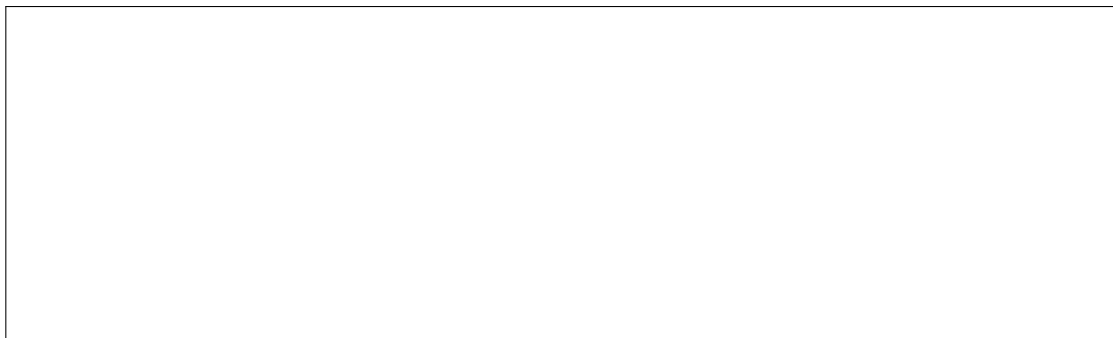


Abbildung 50: Simulierter LoadBalancer über Minikube-Tunnel

Prüfen der externen IP:

```
kubectl get svc nginx-lb
```



Abbildung 51: LoadBalancer mit zugewiesener externer IP-Adresse

```
curl -I http://<EXTERNAL-IP>:8080
```

Listing 3.13: Zugriff über die externe IP



Abbildung 52: Erfolgreicher Zugriff über den LoadBalancer-Service

3.0.8 Vergleich der Service-Typen

- **ClusterIP (Default):** Erzeugt eine virtuelle, *interne* Cluster-IP. Service ist nur *innerhalb* des Clusters erreichbar. Geeignet für Service-zu-Service-Kommunikation (z. B. Backend).
- **NodePort:** Öffnet zusätzlich auf jedem Node einen Port (Standardbereich 30000–32767). Der Service ist *außerhalb* des Clusters via <NodeIP>:<NodePort> erreichbar. Einfach, aber eingeschränkte Flexibilität und keine integrierte L7-Funktionen.
- **LoadBalancer:** Fragt (in Cloud-Umgebungen) einen externen L4-Load-Balancer an und erhält eine *EXTERNAL-IP*. In Minikube wird dies über `minikube service` oder `minikube tunnel` simuliert. Ideal für produktiven Ingress-Traffic pro Service (L4).

Typ	Erreichbarkeit	Einsatzgebiet / Beschreibung
ClusterIP	Nur innerhalb des Clusters	Standardtyp; ermöglicht Kommunikation zwischen Pods oder internen Diensten.
NodePort	Von außen über Node-IP und festen Port erreichbar	Zugriff über <NodeIP>:<NodePort> (z. B. für Tests).
LoadBalancer	Öffentliche IP / Cloud-Load-Balancer	Für produktive Setups; verteilt Traffic über alle Replikas.

Tabelle 1: Kubernetes Service-Typen

3.0.9 YAML-Spezifikationen

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80

```

Listing 3.14: Deployment YAML – nginx

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
  labels:
    app: nginx
spec:

```



```

type: LoadBalancer
selector:
  app: nginx
ports:
  - name: http
    port: 8080
    targetPort: 80

```

Listing 3.15: Service YAML – LoadBalancer

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 4
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip
  labels:
    app: nginx
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - name: http
      port: 8080
      targetPort: 80

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
  labels:
    app: nginx
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - name: http
      port: 8080
      targetPort: 80
      nodePort: 30080
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
  labels:
    app: nginx
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - name: http
      port: 8080
      targetPort: 80

```

Listing 3.16: Kombinierte YAML-Datei (Deployment + alle Service)

3.0.10 Zusammenfassung

- **Cluster:** Ein lokales Kubernetes-Cluster mit Minikube wurde gestartet.
- **Deployment:** Ein nginx-Deployment mit vier Replikas wurde erstellt.
- **Service-Typen:** ClusterIP, NodePort und LoadBalancer wurden erfolgreich getestet.
- **Zugriff:** Über Port-Forwarding, NodePort und Minikube-Tunnel konnte der Webserver erreicht werden.

- **Fazit:** Der Workshop verdeutlicht, wie Deployments und Services in Kubernetes strukturiert sind und welche Service-Typen für interne bzw. externe Zugriffe geeignet sind.